DCUtR 홀 펀칭 과정 분석

개요

DCUtR (Direct Connection Upgrade through Relay)는 libp2p에서 NAT 뒤에 있는 피어들이 릴레이 서버를 통해 직접 연결을 설정할 수 있도록 하는 홀 펀칭 프로토콜입니다. 이는 libp2p의 홀 펀칭 구현체로, 피어 간 직접 통신을 가능하게 합니다.

프로토콜 사양

프로토콜 식별자

- **프로토콜 이름**: `/libp2p/dcutr`
- **메시지 크기 제한**: 4096 바이트

메시지 타입

DCUtR 프로토콜은 두 가지 메시지 타입을 사용합니다:

```
message HolePunch {
enum Type {
CONNECT = 100;
SYNC = 300;
}

required Type type = 1;
```

```
repeated bytes ObsAddrs = 2; // 관찰된 주소들
}
```

1. **CONNECT (100)**: 홀 펀칭 시작 및 주소 교환
2. **SYNC (300)**: 동기화 및 홀 펀칭 완료 확인

아키텍처 구성 요소

1. Behaviour

- **역할**: DCUtR 프로토콜의 최상위 동작 관리
- **주요 기능**:
- 직접 연결 시도 조율
- 릴레이 연결과 직접 연결 매핑 관리
- 주소 후보 관리 (LRU 캐시 사용)
- 최대 3회까지 연결 시도

2. Handler (Relayed)

- **역할**: 릴레이 연결에서의 DCUtR 프로토콜 처리
- **주요 기능**:
- 인바운드/아웃바운드 스트림 관리
- 프로토콜 네고시에이션
- 타임아웃 처리 (10초)

3. Protocol Layer

- **인바운드 핸들러**: 들어오는 홀 펀칭 요청 처리
- **아웃바운드 핸들러**: 나가는 홀 펀칭 요청 처리

홀 펀칭 과정 상세 분석

1. 초기 설정 단계

```
피어 A (Listener) 릴레이 서버 피어 B (Dialer)
| | |
|-- 릴레이 예약 요청 ----->| |
|<-- 예약 승인 ------------| |
| |<-- 릴레이 연결 -------|
|<-- 릴레이 연결 설정 -----|------- 릴레이 연결 ->|
```

2. DCUtR 핸드셰이크 과정

2.1 CONNECT 메시지 교환

피어 B (Dialer) → 피어 A (Listener)

```
// 아웃바운드 핸드셰이크 시작
let msg = proto::HolePunch {
type_pb: proto::Type::CONNECT,
ObsAddrs: candidates.into_iter().map(|a| a.to_vec()).collect(),
};
```

```
stream.send(msg).await?;
```

피어 A (Listener) → 피어 B (Dialer)

```
// 인바운드 핸드셰이크 응답
let msg = proto::HolePunch {
type_pb: proto::Type::CONNECT,
ObsAddrs: candidates.into_iter().map(|a| a.to_vec()).collect(),
};
stream.send(msg).await?;
```

2.2 SYNC 메시지 교환

피어 B (Dialer) → 피어 A (Listener)

```
let msg = proto::HolePunch {
type_pb: proto::Type::SYNC,
ObsAddrs: vec![], // SYNC 메시지에는 주소 없음
};
stream.send(msg).await?;

// RTT의 절반만큼 대기 (타이밍 동기화)
Delay::new(rtt / 2).await;
```

3. 직접 연결 시도

3.1 주소 후보 선택

```
// 주소 후보는 LRU 캐시로 관리되며, 자주 관찰되는 주소가 우선
struct Candidates {
inner: LruCache<Multiaddr, ()>,
me: PeerId,
}

// 릴레이 주소는 필터링됨
fn is_relayed(addr: &Multiaddr) -> bool {
addr.iter().any(|p| p == Protocol::P2pCircuit)
}
```

3.2 동시 다이얼링

```
// 피어 A: 리스너 역할로 다이얼
let opts = DialOpts::peer_id(event_source)
.condition(dial_opts::PeerCondition::Always)
.addresses(remote_addrs)
.override_role() // 역할 오버라이드
.build();

// 피어 B: 다이얼러 역할로 다이얼
let opts = DialOpts::peer_id(event_source)
```

```
.addresses(remote_addrs)

.condition(dial_opts::PeerCondition::Always)

.build();
```

4. 연결 성공 처리

```
// 직접 연결 성공 시

self.queued_events.extend([ToSwarm::GenerateEvent(Event {

remote_peer_id: peer,

result: Ok(connection_id),

})]);
```

핵심 메커니즘

1. 주소 후보 관리

```
impl Candidates {

fn add(&mut self, mut address: Multiaddr) {

if is_relayed(&address) {

return; // 릴레이 주소는 제외

}


if address.iter().last() != Some(Protocol::P2p(self.me)) {
```

```
address.push(Protocol::P2p(self.me));
}

self.inner.push(address, ()); // LRU 캐시에 추가
}
}
```

2. 재시도 메커니즘

```
const MAX_NUMBER_OF_UPGRADE_ATTEMPTS: u8 = 3;

// 연결 실패 시 재시도
if *attempt < MAX_NUMBER_OF_UPGRADE_ATTEMPTS {
self.queued_events.push_back(ToSwarm::NotifyHandler {
handler: NotifyHandler::One(*relayed_connection_id),
peer_id,
event: Either::Left(handler::relayed::Command::Connect),
})
} else {
// 최대 시도 횟수 초과 시 실패 처리
self.queued_events.extend([ToSwarm::GenerateEvent(Event {
remote_peer_id: peer_id,
result: Err(Error {
```

```
inner: InnerError::AttemptsExceeded(MAX_NUMBER_OF_UPGRADE_ATTEMPTS),
}),
})]);
}
```

3. 타이밍 동기화

```
// RTT 측정 및 동기화
let sent_time = Instant::now();
// ... 메시지 교환 ...
let rtt = sent_time.elapsed();

// RTT의 절반만큼 대기하여 동시 다이얼링 타이밍 맞춤
Delay::new(rtt / 2).await;
```

연결 역할 처리

역할 결정 로직

```
// 릴레이 연결에서의 역할과 직접 연결에서의 역할이 반대
match self.endpoint {
ConnectedPoint::Dialer { .. } => {
// 릴레이 연결의 다이얼러는 DCUtR에서 인바운드 스트림 수락
SubstreamProtocol::new(Either::Left(ReadyUpgrade::new(PROTOCOL_NAME)), ())
```

```
}

ConnectedPoint::Listener { .. } => {

// 릴레이 연결의 리스너는 DCUtR에서 아웃바운드 스트림 시작

SubstreamProtocol::new(Either::Right(DeniedUpgrade), ())

}

}
```

오류 처리

주요 오류 타입

```
#[derive(Debug, Error)]

enum InnerError {

#[error("Giving up after {0} dial attempts")]

AttemptsExceeded(u8),

#[error("Inbound stream error: {0}")]

InboundError(protocol::inbound::Error),

#[error("Outbound stream error: {0}")]

OutboundError(protocol::outbound::Error),

}
```

프로토콜 위반 처리

```
#[derive(Debug, Error)]

pub enum ProtocolViolation {

#[error("Expected at least one address in reservation.")]

NoAddresses,

#[error("Unexpected message type 'connect'")]

UnexpectedTypeConnect,

#[error("Unexpected message type 'sync'")]

UnexpectedTypeSync,

}
```

실제 사용 예제

기본 설정

```
#[derive(NetworkBehaviour)]

struct Behaviour {

relay_client: relay::client::Behaviour,

ping: ping::Behaviour,

identify: identify::Behaviour,

dcutr: dcutr::Behaviour, // DCUtR 동작 추가

}


// DCUtR 동작 초기화
```

```
dcutr: dcutr::Behaviour::new(keypair.public().to_peer_id()),
```

리스너 모드

```
// 릴레이를 통한 리스닝
swarm.listen_on(relay_address.with(Protocol::P2pCircuit)).unwrap();

// 예약 요청 승인 대기
SwarmEvent::Behaviour(BehaviourEvent::RelayClient(
relay::client::Event::ReservationReqAccepted { .. },
)) => {
tracing::info!("Relay accepted our reservation request");
}
```

다이얼러 모드

```
// 릴레이를 통한 다이얼링
swarm.dial(
relay_address
.with(Protocol::P2pCircuit)
.with(Protocol::P2p(remote_peer_id)),
).unwrap();
```

```
// DCUtR 이벤트 처리

SwarmEvent::Behaviour(BehaviourEvent::Dcutr(event)) => {

tracing::info!(?event)

}
```

성능 최적화

1. 주소 후보 우선순위

- LRU 캐시를 사용하여 자주 관찰되는 주소를 우선 시도
- 안정적인 포트 매핑을 가진 주소의 성공률이 높음

2. 타임아웃 관리

- 스트림 타임아웃: 10초
- 전체 핸드셰이크 타임아웃 관리
- RTT 기반 동기화로 효율성 향상

3. 리소스 관리

- 동시 스트림 수 제한 (인바운드/아웃바운드 각각 1개)
- 메모리 효율적인 주소 캐시 (최대 20개)

보안 고려사항

1. 주소 검증

- 릴레이 주소 필터링으로 무한 루프 방지

- 유효하지 않은 멀티주소 파싱 오류 처리

2. 리소스 보호

- 최대 재시도 횟수 제한 (3회)
- 스트림 타임아웃으로 리소스 누수 방지
- 동시 연결 시도 수 제한

3. 프로토콜 무결성

- 메시지 타입 검증
- 예상되지 않은 메시지 순서 처리
- 프로토콜 위반 시 적절한 오류 반환

결론

DCUtR은 libp2p에서 NAT 트래버설을 위한 효과적인 홀 펀칭 솔루션을 제공합니다. 릴레이 서버를 통한 초기 연결 설정, 주소 교환, 동시 다이얼링을 통해 직접 P2P 연결을 성공적으로 설정할 수 있습니다. 프로토콜의 단순함과 효율성, 그리고 강력한 오류 처리 메커니즘이 실제 네트워크 환경에서의 안정적인 동작을 보장합니다.